

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')

fear_df = pd.read_csv("C:\\Users\\jpras\\Downloads\\fear_greed_index.csv")
historic_df = pd.read_csv("C:\\Users\\jpras\\Downloads\\historical_data.csv")

#shape
print("Fear greed shape :",fear_df.shape)
print("Historical data shape : ",historic_df.shape)

#missing values
print("\n Missing values in Fear greed dataset :\n",fear_df.isnull().sum())
print("\n Missing values in Historical dataset :\n",historic_df.isnull().sum())

#duplicates
print("\n Duplicate rows in Fear greed dataset :\n",fear_df.duplicated().sum())
print("\n Duplicate rows in Historical dataset :\n",historic_df.duplicated().sum())

#convert timestamps
fear_df['date'] = pd.to_datetime(fear_df['date'])
historic_df['Timestamp'] = pd.to_datetime(historic_df['Timestamp'])

#extract daily date
fear_df['date'] = fear_df['date'].dt.date
historic_df['date'] = historic_df['Timestamp'].dt.date

#merged
merged_df = pd.merge(historic_df,fear_df[['date','classification']], on='date', how='left')
print(merged_df.head())

#key metrics

#PnL per account
daily_pnl = merged_df.groupby(['Account','date'])['Closed PnL'].sum().reset_index()

#win rate per day

```

```

win_rate = merged_df.groupby('date').apply(lambda x: (x['Closed
PnL']>0).mean()).reset_index(name='win_rate')

#avg trade size

avg_trade_size = merged_df.groupby('date')['Size USD'].mean().reset_index(name =
'avg_trade_size_usd')

#leverage distribution

leverage_distribution = merged_df.groupby('date')['Execution Price'].describe()

#no of trades per day

trade_count = merged_df.groupby('date').size().reset_index(name='num_trades')

#long/short ratio per day

long_short_ratio = merged_df.groupby(['date','Side']).size().unstack(fill_value=0)

long_short_ratio['long_short_ratio'] = long_short_ratio.get('Long',0)
/(long_short_ratio.get('Short',1))

# Preview results

print("\nDaily PnL per account:\n", daily_pnl.head())

print("\nWin rate per day:\n", win_rate.head())

print("\nAverage trade size (USD) per day:\n", avg_trade_size.head())

print("\nLeverage distribution per day:\n", leverage_distribution.head())

print("\nNumber of trades per day:\n", trade_count.head())

print("\nLong/Short ratio per day:\n", long_short_ratio.head())

```

```

Fear greed shape : (2644, 4)
Historical data shape : (211224, 16)

Missing values in Fear greed dataset :
timestamp      0
value          0
classification  0
date           0
dtype: int64

Missing values in Historical dataset :
Account        0
Coin           0
Execution Price 0
Size Tokens    0
Size USD       0
Side           0
Timestamp IST   0
Start Position  0
Direction      0
Closed PnL     0
Transaction Hash 0
Order ID       0
Crossed        0
Fee            0
Trade ID       0
Timestamp      0
dtype: int64

Duplicate rows in Fear greed dataset :
0

Duplicate rows in Historical dataset :
0

   Account      Coin  ...      date  classification
0  0xae5eacaf9c6b9111fd53034a602c192a04e082ed  @107  ...  1970-01-01      NaN
1  0xae5eacaf9c6b9111fd53034a602c192a04e082ed  @107  ...  1970-01-01      NaN
2  0xae5eacaf9c6b9111fd53034a602c192a04e082ed  @107  ...  1970-01-01      NaN
3  0xae5eacaf9c6b9111fd53034a602c192a04e082ed  @107  ...  1970-01-01      NaN
4  0xae5eacaf9c6b9111fd53034a602c192a04e082ed  @107  ...  1970-01-01      NaN

[5 rows x 18 columns]

```

```

Daily PnL per account:
      Account      date      Closed PnL
0  0x083384f897ee0f19899168e3b1bec365f52a9012  1970-01-01  1.600230e+06
1  0x23e7a7f8d14b550961925fbfdaa92f5d195ba5bd  1970-01-01  4.788532e+04
2  0x271b280974205ca63b716753467d5a371de622ab  1970-01-01 -7.043619e+04
3  0x28736f43f1e871e6aa8b1148d38d4994275d72c4  1970-01-01  1.324648e+05
4  0x2c229d22b100a7beb69122eed721cee9b24011dd  1970-01-01  1.686580e+05

Win rate per day:
      date  win_rate
0  1970-01-01  0.411265

Average trade size (USD) per day:
      date  avg_trade_size_usd
0  1970-01-01      5639.45121

Leverage distribution per day:
      date      count      mean      std      ...      50%      75%      max
1970-01-01  211224.0  11414.72335  29447.654868  ...  18.28  101.58  109004.0

[1 rows x 8 columns]

Number of trades per day:
      date  num_trades
0  1970-01-01      211224

Long/Short ratio per day:
      Side      BUY      SELL  long_short_ratio
date
1970-01-01  102696  108528      0.0

```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
sns.set(style="whitegrid")
```

```
# Sentiment dataset: has a 'date' column
```

```
sentiment_df = pd.read_csv("C:\\Users\\jpras\\Downloads\\fear_greed_index.csv",
parse_dates=["date"])
```

```
sentiment_df['date'] = sentiment_df['date'].dt.date
```

```
traders_df = pd.read_csv("C:\\Users\\jpras\\Downloads\\historical_data.csv")
```

```
traders_df['date'] = pd.to_datetime(
    traders_df['Timestamp IST'],
    format="%d-%m-%Y %H:%M", # matches '18-03-2025 12:50'
    errors="coerce"
```

```
).dt.date
```

```
# Merge datasets
```

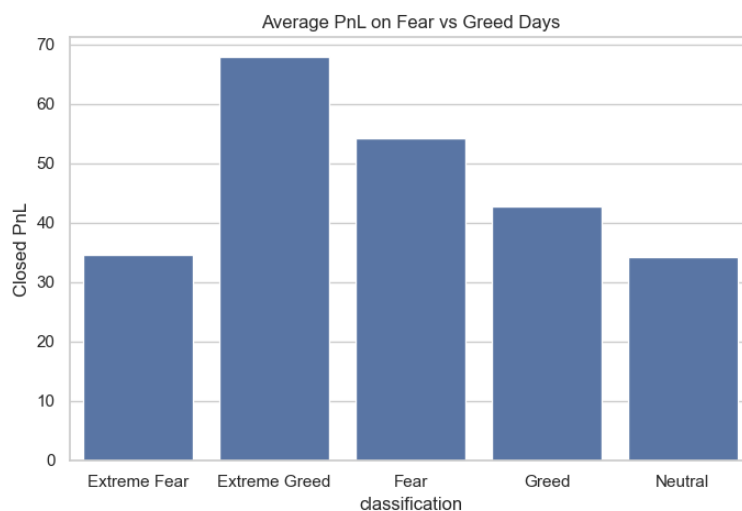
```
merged_df = pd.merge(
    traders_df,
    sentiment_df[['date','classification']],
    on="date",
```

```

    how="left"
)
merged_df['win'] = (merged_df['Closed PnL'] > 0).astype(int)
#1Pnl
merged_df['win'] = (merged_df['Closed PnL'] > 0).astype(int)

# Example drawdown proxy: daily min PnL per account
drawdown_df = merged_df.groupby(['date','Account'])['Closed PnL'].min().reset_index()
drawdown_df.rename(columns={'Closed PnL':'drawdown_proxy'}, inplace=True)
merged_df = merged_df.merge(drawdown_df, on=['date','Account'], how='left')
performance = merged_df.groupby('classification').agg({
    'Closed PnL':'mean',
    'win':'mean',
    'drawdown_proxy':'mean'
}).reset_index()
print(performance)
#1 Pnl
plt.figure(figsize=(8,5))
sns.barplot(data=performance, x='classification', y='Closed PnL')
plt.title("Average PnL on Fear vs Greed Days")
plt.show()

```



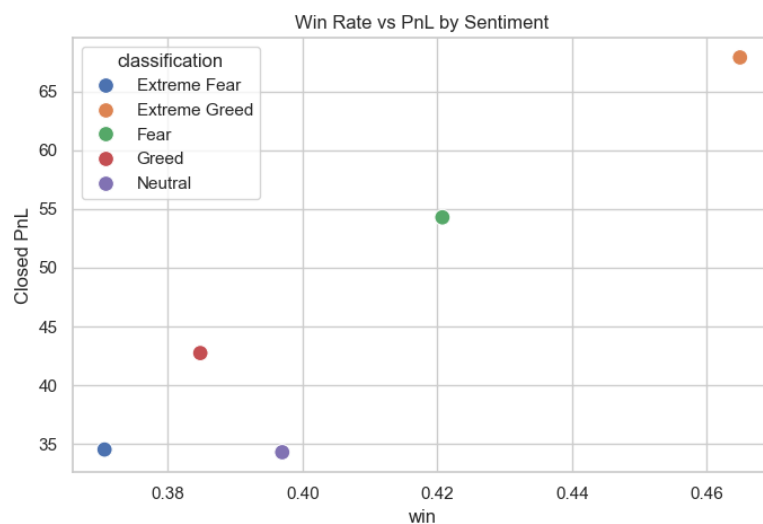
#1 win rate

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(data=performance, x='win', y='Closed PnL', hue='classification', s=100)
```

```
plt.title("Win Rate vs PnL by Sentiment")
```

```
plt.show()
```



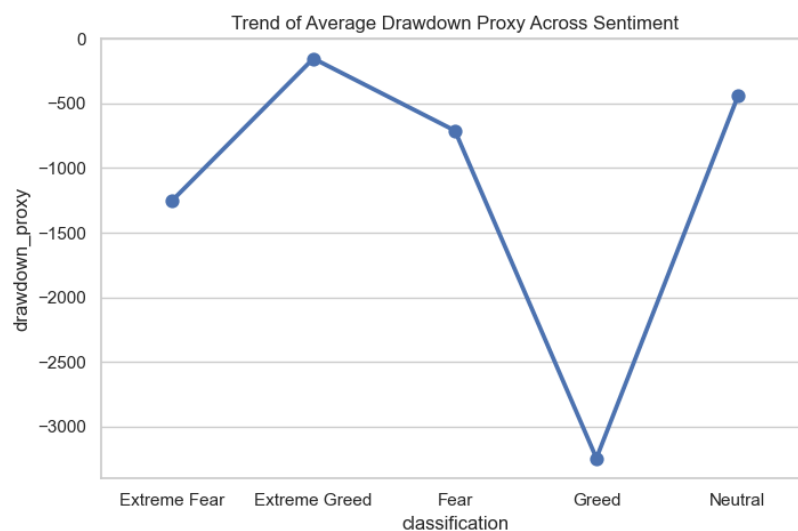
#1 drawdown proxy

```
plt.figure(figsize=(8,5))
```

```
sns.pointplot(data=performance, x='classification', y='drawdown_proxy', markers='o', linestyle='-')
```

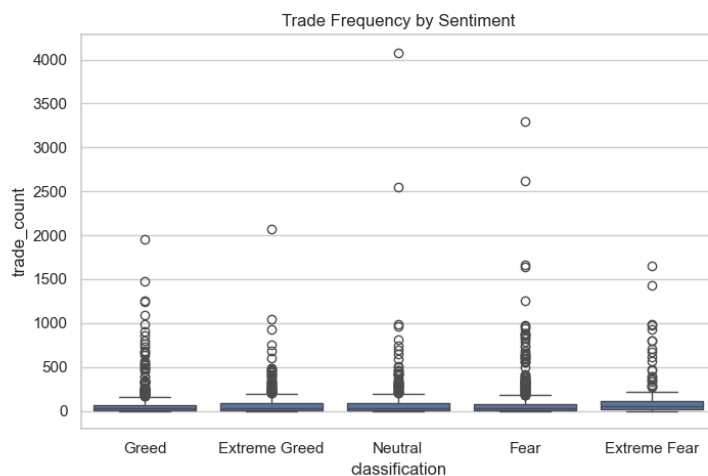
```
plt.title("Trend of Average Drawdown Proxy Across Sentiment")
```

```
plt.show()
```



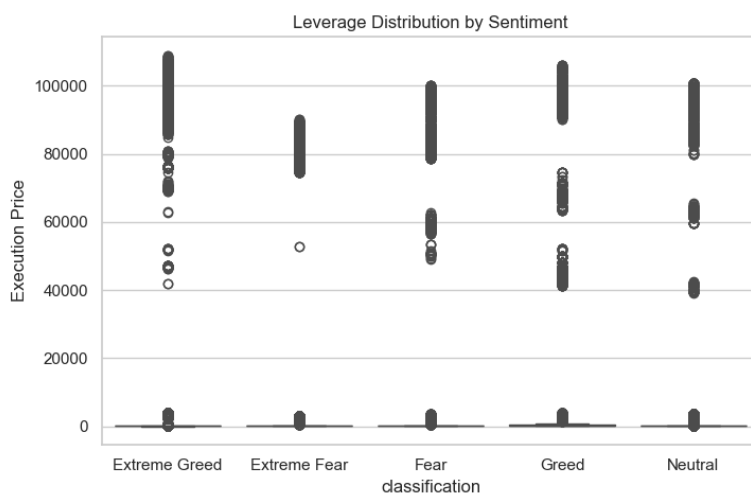
#2 trade frequency

```
trade_freq =  
merged_df.groupby(['date','classification','Account']).size().reset_index(name='trade_count')  
  
plt.figure(figsize=(8,5))  
  
sns.boxplot(data=trade_freq, x='classification', y='trade_count')  
  
plt.title("Trade Frequency by Sentiment")  
  
plt.show()
```



#2 leverage

```
plt.figure(figsize=(8,5))  
  
sns.boxplot(data=merged_df, x='classification', y='Execution Price')  
  
plt.title("Leverage Distribution by Sentiment")  
  
plt.show()
```



#2 long/short bias

```

long_short = merged_df.groupby(['classification', 'Side']).size().reset_index(name='count')
long_short_pivot = long_short.pivot(index='classification', columns='Side', values='count').fillna(0)

# Adjust ratio calculation to match actual column names
long_short_pivot['long_ratio'] = long_short_pivot['BUY'] / (long_short_pivot['BUY'] +
long_short_pivot['SELL'])

print(long_short_pivot)

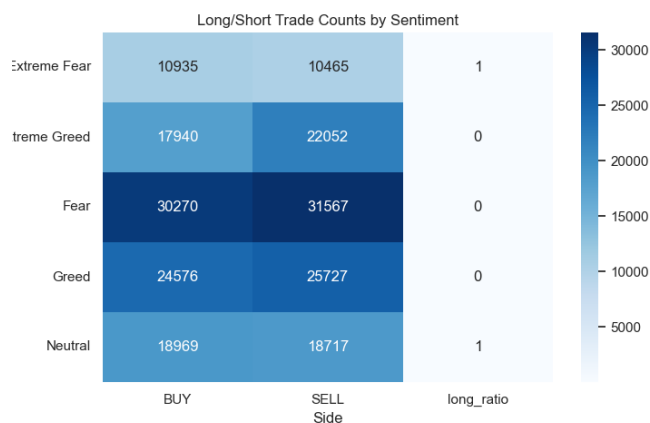
plt.figure(figsize=(8,5))

sns.heatmap(long_short_pivot, annot=True, fmt=".0f", cmap="Blues")

plt.title("Long/Short Trade Counts by Sentiment")

plt.show()

```



#2 position sizes

```

pos_size = merged_df.groupby('classification')['Size USD'].mean().reset_index()

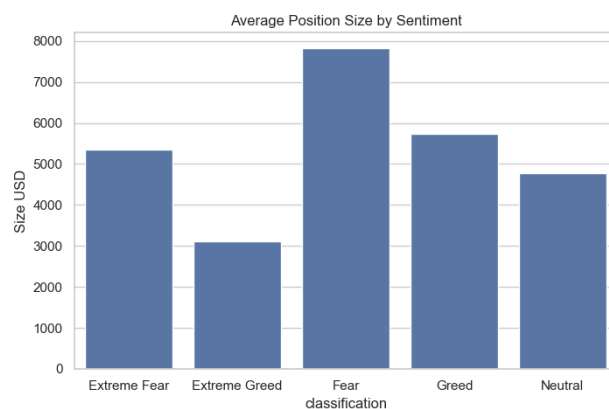
plt.figure(figsize=(8,5))

sns.barplot(data=pos_size, x='classification', y='Size USD')

plt.title("Average Position Size by Sentiment")

plt.show()

```

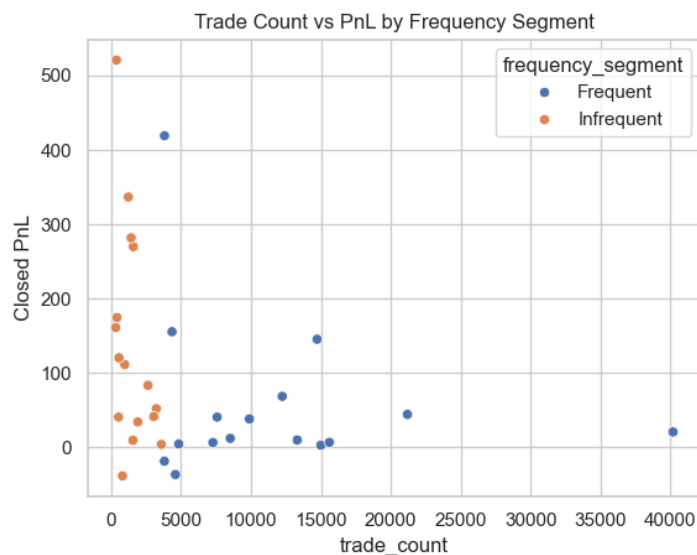


#3 frequent vs infrequent

```
avg_pnl = merged_df.groupby('Account')['Closed PnL'].mean().reset_index()
trade_counts = merged_df.groupby('Account').size().reset_index(name='trade_count')
account_perf = avg_pnl.merge(trade_counts, on='Account')

account_perf['frequency_segment'] = account_perf['trade_count'].apply(
    lambda x: 'Frequent' if x > account_perf['trade_count'].median() else 'Infrequent'
)

sns.scatterplot(data=account_perf, x='trade_count', y='Closed PnL', hue='frequency_segment')
plt.title("Trade Count vs PnL by Frequency Segment")
plt.show()
```



#3 Consistent winners vs Inconsistent traders

```
win_rates = merged_df.groupby('Account')['win'].mean().reset_index(name='win_rate')
merged_df = merged_df.merge(win_rates, on='Account', how='left')
merged_df['consistency_segment'] = merged_df['win_rate'].apply(lambda x: 'Consistent' if x > 0.55
else 'Inconsistent')

consistency_perf = merged_df.groupby('consistency_segment').agg({
```



```

    'Closed PnL': 'mean',
    'Execution Price': 'mean'
}).reset_index()

print(consistency_perf)

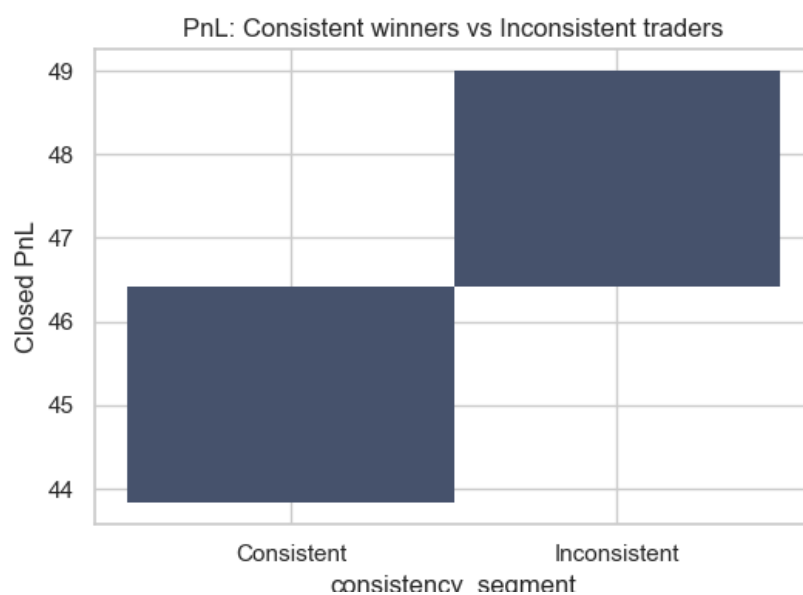
plt.figure(figsize=(6,4))

sns.histplot(data=consistency_perf, x='consistency_segment', y='Closed PnL')

plt.title("PnL: Consistent winners vs Inconsistent traders")

plt.show()

```



#3 high leverage vs low leverage traders

```

median_size = merged_df['Size USD'].median()

merged_df['leverage_segment'] = merged_df['Size USD'].apply(
    lambda x: 'High' if x > median_size else 'Low'
)

print(merged_df['leverage_segment'].value_counts())

leverage_perf = merged_df.groupby('leverage_segment').agg({
    'Closed PnL': 'mean',
    'win': 'mean',
    'Size USD': 'mean'
}).reset_index()

```

```

print(leverage_perf)

sns.heatmap(leverage_perf.set_index('leverage_segment'), annot=True, cmap="coolwarm",
fmt=".2f")

plt.title("Performance Metrics: High vs Low Leverage Traders")

plt.show()

```

